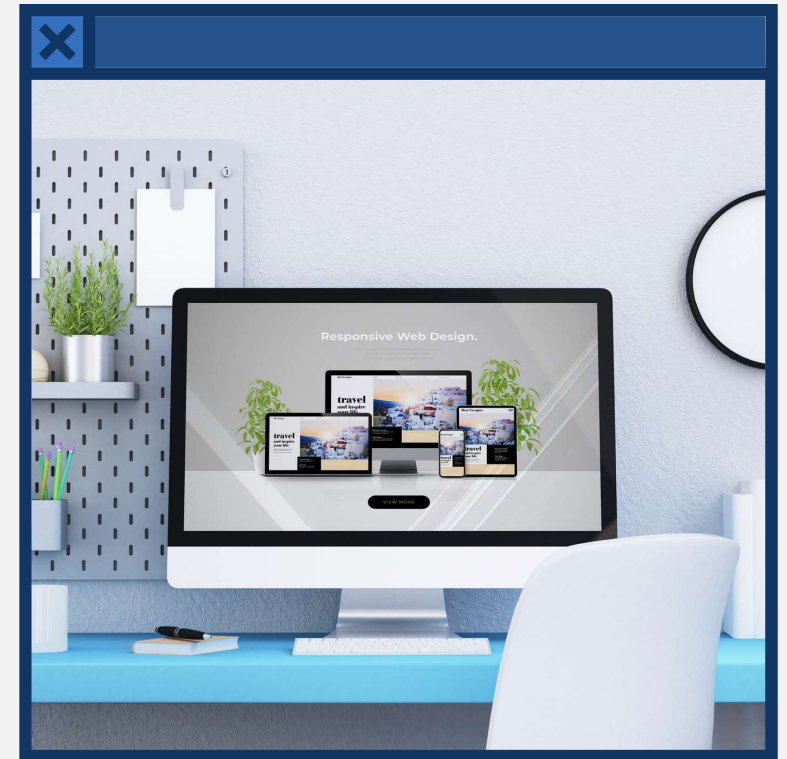


LAYOUT - 404

TECNOLOGÍAS WEB II

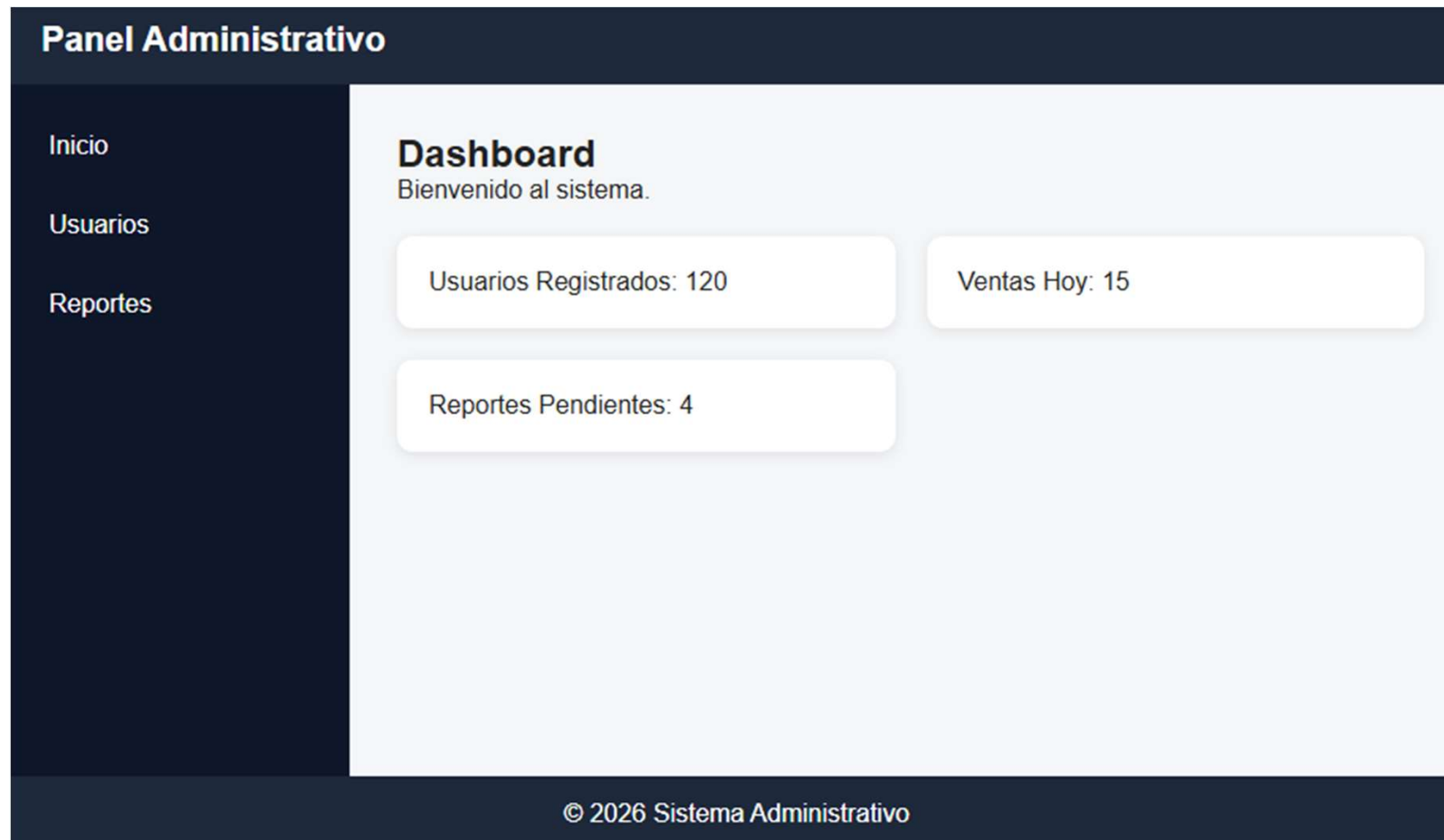
M. SC. ING. PÁVEL FRANZ CÁCERES TÓRREZ



Plantilla html y css



Verificamos Template Admin



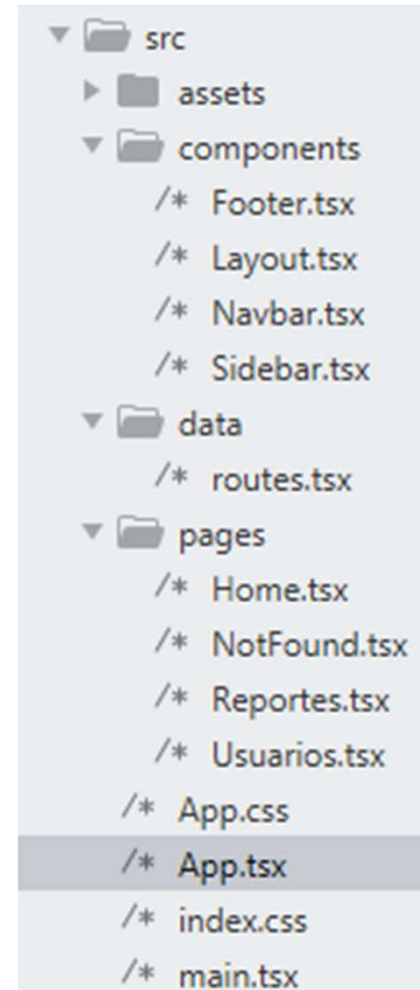
Construyendo Layout y 404 en React



Paso 1: Instalando proyecto y react-router-dom

```
λ npm create vite@latest mi-app -- --template react-ts  
  
> npx  
> create-vite mi-app --template react-ts  
  
◇ Install with npm and start now?  
  Yes
```

```
λ npm install react-router-dom  
  
added 4 packages, and audited 158 packages in 4s  
  
43 packages are looking for funding  
  run `npm fund` for details  
  
found 0 vulnerabilities
```



```
▼ src  
  ► assets  
  ▼ components  
    /* Footer.tsx  
    /* Layout.tsx  
    /* Navbar.tsx  
    /* Sidebar.tsx  
  ▼ data  
    /* routes.tsx  
  ▼ pages  
    /* Home.tsx  
    /* NotFound.tsx  
    /* Reportes.tsx  
    /* Usuarios.tsx  
  /* App.css  
  /* App.tsx  
  /* index.css  
  /* main.tsx
```

Paso 2: Layout - Identificamos componentes que se repiten para llevar a components

```
index.html x reportes.html x usuarios.html
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4  <meta charset="UTF-8">
5  <meta name="viewport" content="width=device-width, initial-scale=1">
6  <title>Inicio</title>
7  <link rel="stylesheet" href="styles.css">
8  </head>
9  <body>
10
11  <header class="header"> ...
13  </header>
14
15  <div class="container">
16
17  <aside class="sidebar"> ...
23  </aside>
24
25  <main class="main"> ...
34  </main>
35
36  </div>
37
38  <footer class="footer"> ...
40  </footer>
41
42  </body>
43  </html>
```

components/

Navbar.tsx (header)

Sidebar.tsx

Footer.tsx

Layout.tsx

Paso 3: Establecimiento de la estructura del proyecto

src/

components/

Navbar.tsx

Sidebar.tsx

Footer.tsx

Layout.tsx

Header

Menu de navegacion

Pie de página

Maquetación

pages/

Home.tsx

Usuarios.tsx

Reportes.tsx

NotFound.tsx

Página Home

Página Usuarios

Página Reportes

404

data/

routes.tsx

Archivo de rutas

App.tsx

App.css

index.css

Componente raíz

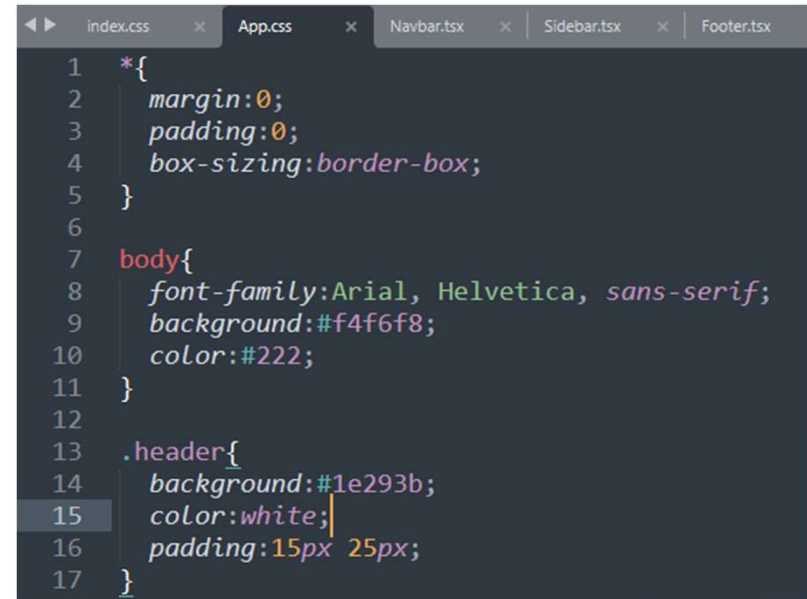
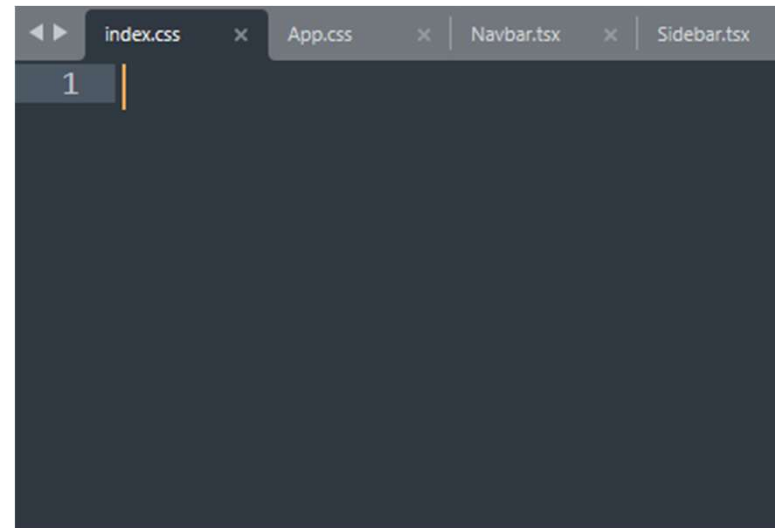
Estilos de la app

Estilos globales

Paso 4: Tratamiento de estilos

- **Index.css:** Sin estilos globales
- **App.css** Todo el CSS del dashboard:
 - header
 - sidebar
 - cards
 - footer
 - table
 - responsive

Esto debe ir inicialmente
porque ayuda a visualizar.



Paso 5: Páginas

Las rutas necesitan páginas reales

Home.tsx

```
index.css  App.css  Home.tsx  Navbar.tsx  Sidebar.tsx  Foc
1  export default function Home() {
2    return (
3      <>
4        <h2>Dashboard</h2>
5        <p>Bienvenido al sistema.</p>
6
7        <div className="cards">
8          <div className="card">Usuarios: 120</div>
9          <div className="card">Ventas: 35</div>
10         <div className="card">Reportes: 8</div>
11        </div>
12      </>
13    );
14  }
```

Card es un componente,
pero en esta sesión
Interesa:

Layout + Router + 404

Usuarios.tsx

```
1  const usuarios = [
2    { id: 1, nombre: "Carlos", email: "carlos@mail.com" },
3    { id: 2, nombre: "María", email: "maria@mail.com" }
4  ];
5
6  export default function Usuarios() {
7    return (
8      <>
9        <h2>Usuarios</h2>
10
11        <table>
12          <thead>
13            <tr>
14              <th>ID</th>
15              <th>Nombre</th>
16              <th>Email</th>
17            </tr>
18          </thead>
19
20          <tbody>
21            {usuarios.map((u) => (
22              <tr key={u.id}>
23                <td>{u.id}</td>
24                <td>{u.nombre}</td>
25                <td>{u.email}</td>
26              </tr>
27            ))}
28          </tbody>
29        </table>
30      </>
31    );
32  }
```

Se crea un arreglo de objetos llamado usuarios.

Cada usuario con: id nombre email

Esto simula datos que más adelante podrían venir desde una **API/base de datos**.

Reportes.tsx

```
index.css × App.css × Home.tsx × Usuarios.tsx × Reportes.tsx × Navbar.tsx × Sid
1  export default function Reportes() {
2    return (
3      <>
4        <h2>Reportes</h2>
5
6        <div className="cards">
7          <div className="card">Reporte Ventas Mensuales</div>
8          <div className="card">Reporte Usuarios Activos</div>
9          <div className="card">Reporte Inventario</div>
10       </div>
11     </>
12   );
13 }
```

En base a la estructura que esta en el template.

NotFound.tsx

```
1 import { Link } from "react-router-dom";
2
3 export default function NotFound() {
4   return (
5     <>
6       <h1>404</h1>
7       <p>Página no encontrada</p>
8       <br />
9       <Link to="/">Volver al Inicio</Link>
10    </>
11  );
12 }
```

- Página error 404
- Se importa Link desde react-router-dom.
- Link sirve para navegar entre rutas dentro de React

Paso 6: Routes.tsx

Configuración de las rutas del sistema

Routes.tsx

```
routes.tsx x NotFound.tsx x Reportes.tsx x Usuarios.tsx x Home.tsx
1 import Home from "../pages/Home";
2 import Usuarios from "../pages/Usuarios";
3 import Reportes from "../pages/Reportes";
4
5 export const routes = [
6   {
7     name: "Inicio",
8     path: "/",
9     element: <Home />
10  },
11  {
12    name: "Usuarios",
13    path: "/usuarios",
14    element: <Usuarios />
15  },
16  {
17    name: "Reportes",
18    path: "/reportes",
19    element: <Reportes />
20  }
21 ];
```

Importamos las páginas
Creamos un arreglo routes con:
name
Path
element

Paso 7: Creamos componentes

Navbar.tsx

```
Navbar.tsx x Footer.tsx x routes.tsx x NotFound.tsx x Reportes.tsx x Usuarios.tsx
1  type NavbarProps = {
2    title: string;
3  };
4
5  export default function Navbar({ title }: NavbarProps) {
6    return (
7      <header className="header">
8        <h1>{title}</h1>
9      </header>
10   );
11 }
```

Footer.tsx

```
Navbar.tsx x Footer.tsx x routes.tsx x NotFound.tsx x Reportes.tsx x Usuarios.tsx x Home.ts
1  type FooterProps = {
2    company: string;
3    year: number;
4  };
5
6  export default function Footer({ company, year }: FooterProps) {
7    return (
8      <footer className="footer">
9        © {year} {company}
10     </footer>
11   );
12 }
```

Sidebar.tsx

```
1 import { Link } from "react-router-dom";
2
3 type MenuItem = {
4   name: string;
5   path: string;
6 };
7
8 type SidebarProps = {
9   menu: MenuItem[];
10 };
11
12 export default function Sidebar({ menu }: SidebarProps) {
13   return (
14     <aside className="sidebar">
15       <ul>
16         {menu.map((item, index) => (
17           <li key={index}>
18             <Link to={item.path}>{item.name}</Link>
19           </li>
20         ))}
21       </ul>
22     </aside>
23   );
24 }
```

MenuItem en Sidebar.tsx no necesita element porque el Sidebar solo muestra enlaces visuales, no renderiza páginas.

Mientras que routes.tsx sí necesita element porque define qué componente cargar cuando visitas una ruta.

Crearé algo así

`<Link to="/usuarios">Usuarios</Link>`.

Layout.tsx (Une todo)

```
Layout.tsx x Sidebar.tsx x Navbar.tsx x Footer.tsx x routes.tsx x NotFound.tsx x Repo
1 import { Outlet } from "react-router-dom";
2
3 import Navbar from "../Navbar";
4 import Sidebar from "../Sidebar";
5 import Footer from "../Footer";
6
7 import { routes } from "../data/routes";
8
9 export default function Layout() {
10   const menu = routes.map((route) => ({
11     name: route.name,
12     path: route.path
13   }));
14
15   return (
16     <>
17       <Navbar title="Panel Administrativo" />
18       <div className="container">
19         <Sidebar menu={menu} />
20         <main className="main">
21           <Outlet />
22         </main>
23       </div>
24       <Footer company="Sistema Administrativo" year={2026} />
25     </>
26   );
27 }
```

<Outlet /> es un componente de react-router-dom que actúa como contenedor dinámico para mostrar las rutas hijas dentro del layout.

Navbar, Sidebar y Footer quedan fijos. Solo cambia el contenido del <Outlet />.

Se importa routes (Donde existen las rutas y menu)

menu, lista para mostrar botones

Paso 8: App.tsx

```
App.tsx | Layout.tsx | Sidebar.tsx | Navbar.tsx | Footer.tsx | routes.tsx | NotFound.tsx | Reportes.tsx
1  import './App.css';
2  import { BrowserRouter, Routes, Route } from 'react-router-dom';
3
4  import Layout from './components/Layout';
5  import NotFound from './pages/NotFound';
6
7  import { routes } from './data/routes';
8
9  export default function App() {
10   return (
11     <BrowserRouter>
12       <Routes>
13
14         <Route path="/" element={<Layout />}>
15
16           {routes.map((route, index) => (
17             <Route
18               key={index}
19               index={route.path === "/" ? undefined : route.path.slice(1)}
20               path={route.path === "/" ? undefined : route.path.slice(1)}
21               element={route.element}
22             />
23           ))}
24
25         </Route>
26
27         <Route path="*" element={<NotFound />} />
28
29       </Routes>
30     </BrowserRouter>
31   );
32 }
```

<BrowserRouter> contenedor principal del sistema de rutas.

Routes contenedor donde declaras todas las rutas posibles. (Dentro van varios <Route />.)

Route construye

<Route path="/usuarios" element={<Usuarios />} />

Si la URL es /usuarios, mostrar componente Usuarios.

Solo importamos Layout por que Las páginas internas (Home, Usuarios, Reportes) se mostrarán dentro de: <Outlet /> que está en Layout.tsx.

Layout es la ruta padre (contenedora) de todas esas páginas internas.

<Route path="/" element={<Layout />}>

...rutas hijas...

</Route>

El método map() recorre el arreglo routes y genera automáticamente las rutas hijas.

Generará algo así

<Route index element={<Home />} />

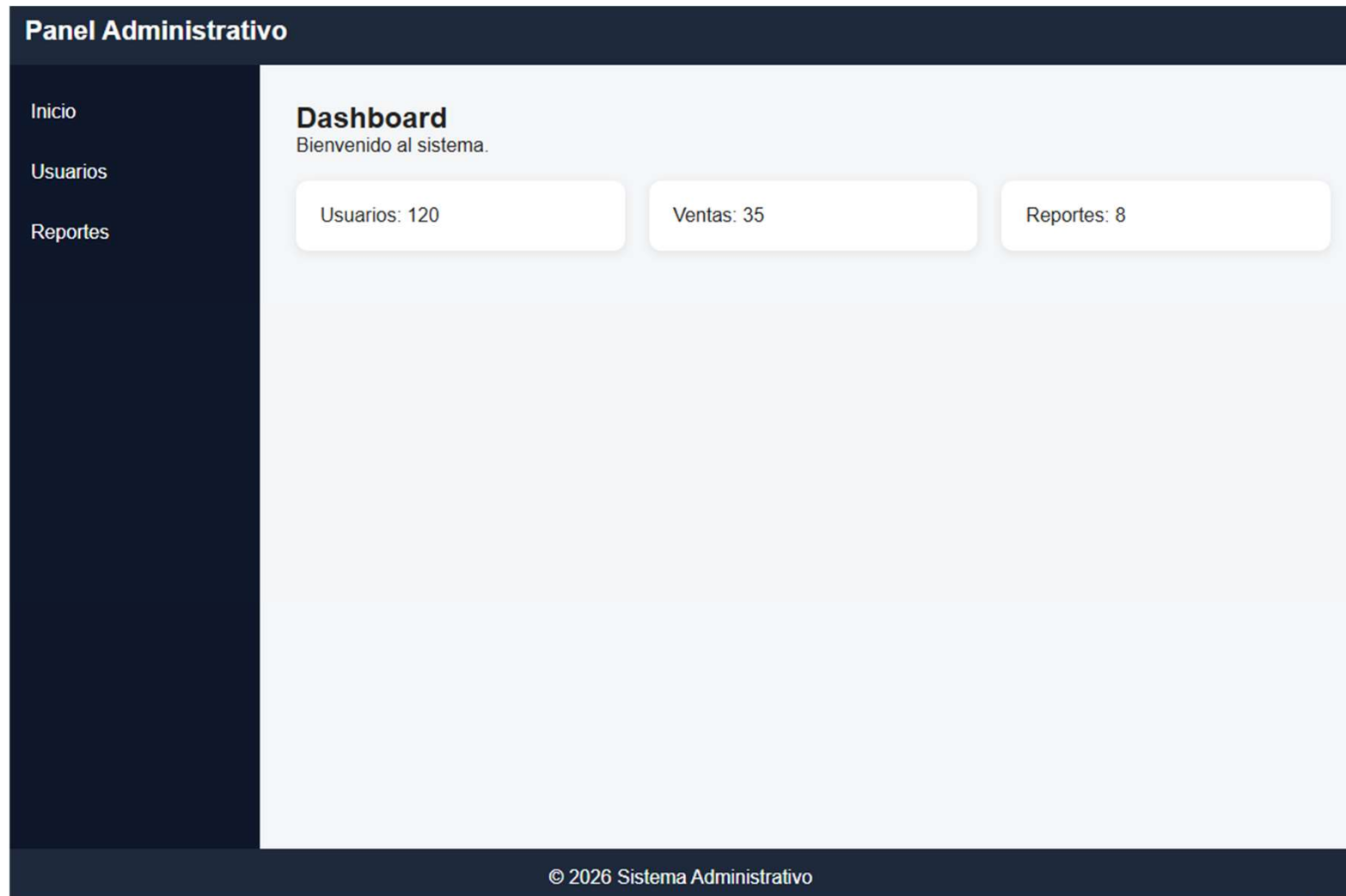
<Route path="usuarios" element={<Usuarios />} />

<Route path="reportes" element={<Reportes />} />

con

```
{routes.map((route, index) => (
  <Route
    key={index}
    index={route.path === "/" ? undefined : route.path.slice(1)}
    path={route.path === "/" ? undefined : route.path.slice(1)}
    element={route.element}
  />
))}
```

Probar Panel Administrativo



Agregando una nueva página



Paso 1: Creamos la pagina

Roles.tsx

```
Roles.tsx x App.tsx x Layout.tsx x Sidebar.tsx x Navbar.tsx x Footer.tsx x rou
1  export default function Roles() {
2    const roles = ["Administrador", "Editor", "Usuario"];
3
4    return (
5      <>
6        <h2>Roles</h2>
7        <p>Listado de roles del sistema.</p>
8
9        <ul>
10         {roles.map((rol, index) => (
11           <li key={index}>{rol}</li>
12         ))}
13       </ul>
14     </>
15   );
16 }
```

```
export default function Roles() {
  const roles = ["Administrador", "Editor", "Usuario"];

  return (
    <>
      <h2>Roles</h2>
      <p>Listado de roles del sistema.</p>

      <ul>
        {roles.map((rol, index) => (
          <li key={index}>{rol}</li>
        ))}
      </ul>
    </>
  );
}
```

Paso 2: Importar en routes.tsx

```
routes.tsx  Roles.tsx  App.tsx  Layout.tsx  Sidebar.tsx
1  import Home from "../pages/Home";
2  import Usuarios from "../pages/Usuarios";
3  import Reportes from "../pages/Reportes";
4  import Roles from "../pages/Roles";
5
6  export const routes = [
7    {
8      name: "Inicio",
9      path: "/",
10     element: <Home />
11   },
12   {
13     name: "Usuarios",
14     path: "/usuarios",
15     element: <Usuarios />
16   },
17   {
18     name: "Reportes",
19     path: "/reportes",
20     element: <Reportes />
21   }
22 ];
```

import Roles from "../pages/Roles";

Paso 3: Agregando al arreglo routes

```
routes.tsx
1 import Home from "../pages/Home";
2 import Usuarios from "../pages/Usuarios";
3 import Reportes from "../pages/Reportes";
4 import Roles from "../pages/Roles";
5
6 export const routes = [
7   {
8     name: "Inicio",
9     path: "/",
10    element: <Home />
11  },
12  {
13    name: "Usuarios",
14    path: "/usuarios",
15    element: <Usuarios />
16  },
17  {
18    name: "Roles",
19    path: "/roles",
20    element: <Roles />
21  },
22  {
23    name: "Reportes",
24    path: "/reportes",
25    element: <Reportes />
26  }
27 ];
```

```
{
  name: "Roles",
  path: "/roles",
  element: <Roles />
},
```

Panel Administrativo

Inicio

Usuarios

Roles

Reportes

Roles

Listado de roles del sistema.

- Administrador
- Editor
- Usuario

Ejercicio





GRACIAS

